```python
import math
from numbers import Number
import numpy as np
import sys
```

Q1

```python
def simulate_single_server(T, lmda, g):
    '''
    Simulating M/G/1/infinity/FIFO queue
    Args:
        T (float): total time that the system is alive (closing time
of the system assuming start time = 0)
        lmda (float): rate of arrival into the system
        g: distribution of the service time
    '''

    def get_Ts(s, lmda):
        u = np.random.rand()
        Ts = s+1/lmda*math.log(1/u)
        return Ts

    def generate_Y(g):
        h = lambda x: math.exp(-x)
        xstar = 2/39 # computed on paper (where (g/h)' = 0)
        c = g(xstar)/h(xstar)
        while(True):
            u1 = np.random.rand()
            l = -math.log(u1)
            u2 = np.random.rand()
            if(u2 <= (g(l)/h(l))/c):
                y = l
                break
        return y

    # Start time (0), End time (T), Arrival distribution, and Service
distribution are specific to the system (they don't change with
iterations)
    # Initialization
    t = 0
    n = 0
    Na = 0 # Number of arrivals
    A = [] # Arrival times
    Nd = 0 # Number of departures
    D = [] # Departure times
    T0 = get_Ts(0, lmda)
    ta = T0
    td = sys.maxsize # to model infinity
    tidle = 0 # how long the system is idle
```

```
    # Running the system
    while(True):
        if(ta<=td and ta<=T): # ta = min(ta, td, T)
            if(n==0): # for idle time, add difference between current
time and time of next arrival
                tidle += (ta-t)
                # Here we add time only when someone arrives; if no
one arrives till the end, we need to account for that also
                # That will happen when n==0 and no ones arrives till
the end => T<next ta => added in the last elif
            t = ta
            Na = Na+1
            n = n+1
            Tt = get_Ts(t, lmda) # generate Tt
            ta = Tt
            if(n==1):
                Y = generate_Y(g)
                td = t+Y
            A.append(t)# A(Na) = t # A should be a vector

        elif(td<ta and td<=T):
            t = td
            n = n-1
            Nd = Nd+1
            if(n==0):
                td = sys.maxsize
            else:
                Y = generate_Y(g)
                td = t+Y
            D.append(t) # D(Nd) = t # so should be D
        elif(T<min(ta, td) and n>0): # ta>T => no more arrivals
            t = td
            n = n-1
            Nd = Nd+1
            if(n>0):
                Y = generate_Y(g)
                td = t+Y
            D.append(t) # D(Nd) = t
        elif(T<min(ta, td) and n==0):
            tidle += max(T-t, 0)
            Tp = max(t-T, 0)
            break

    return A, D, Tp, tidle

T = 9
lmda = 10
def g(x):
    return 20*math.exp(-40*x)*(40*x)**2
```

```python
for num_iters in [100, 1000]:
    print(num_iters, "iterations")
    cnt = 0 # number of customers who entered the system (over all iterations)
    Tsys_sum = 0 # average time a customer spends in the system
    Tp_sum = 0 # average delay (time past T) that the server has to stay awake
    for _ in range(num_iters):
        A, D, Tp, _ = simulate_single_server(T, lmda, g)
        assert(len(A) == len(D)) # The one who entered has to exit
        cnt += len(A)
        for i in range(len(A)):
            Tsys_sum += D[i] - A[i]
        Tp_sum += Tp

    Tsys_mean = Tsys_sum/cnt
    Tp_mean = Tp_sum/num_iters

    print("Avg customer time in the system:", Tsys_mean)
    print("Avg extension time of the system:", Tp_mean)
```

```
100 iterations
Avg customer time in the system: 0.21011168622531998
Avg extension time of the system: 0.15016975619152867
1000 iterations
Avg customer time in the system: 0.20940307876792727
Avg extension time of the system: 0.14495869478801873
```

Q2

```python
T = 1 # idle time in a [day]
lmda = 10
def g(x):
    return 20*math.exp(-40*x)*(40*x)**2

num_iters = 1000
t_idle_sum = 0
for _ in range(num_iters):
    _, _, _, t_idle = simulate_single_server(T, lmda, g)
    t_idle_sum += t_idle

t_idle_mean = t_idle_sum/num_iters
print("Avg time, the system is idle:", t_idle_mean)
```

```
Avg time, the system is idle: 0.36715362649616773
```

Q5

Updating procedures (pseudocode)

```
def process_single_server_with_waittime():
    # Initialize (state) variables
    t = 0
    Na = 0
    n = 0
    Nl = 0
    Nd = 0
    Avec = [0] # stores the (absolute) arrival time of each person
    Wvec = [0] # stores the (absolute) waiting time for each person
(the (absolute) time till which that person will wait)
    Dvec = [0] # stored the (absolute) departing time for each person
(infinity if that person or job was lost)
    person_served = 0 # represents the most recent person that was
served (his/her index as per their order of arrival; no person served
till now)

    # Initialize time variables
    delta = generate_time_for_next_arrival()
    ta = t + delta
    tl = sys.maxsize
    td = sys.maxsize

    # Event loop
    while(True):
        te = min(ta, td, tl)
        if(te > T):
            return Nl # number of people (or jobs) lost till time T
        elif(te == ta): # next event is an arrival
            t = te
            # Update for arrivals
            Na += 1
            Avec.append(t)
            # Update for presence in the system
            n += 1
            # Update for wait times
            delta = generate_waittime()
            tlp = t + delta
            Wvec.append(tlp)
            assert(len(Wvec) == len(Avec))
            tl = min(tl, tlp)
            # Update for departures
            if(n == 1):
                # would the condition have Na or n? If this person
drops before getting served, system is again empty =>
                # need to generate time again => generate when n = 1
(there is only one person in the system irrespective of how many
```

```
arrived till then)
                delta = generate_service_time_for_next_job()
                td = t + delta
                Dvec.append(td)
            else:
                Dvec.append(sys.maxsize) # Not sure at which time this
person will be served (since this person can be lost), so infinity
            assert(len(Dvec) == len(Wvec))
            # Update for the next arrival
            delta = generate_time_for_next_arrival()
            ta = t + delta
        elif(te == tl): # next event is loss of a job
            t = te
            # Dropping the person whose wait time ended and finding
the next (min) wait time
            for i in range(person_served+1, len(Wvec)):
                if(Wvec[i] == t): # this was the person whose wait
time ended and who got lost
                    Wvec[i] = sys.maxsize
                    n -= 1
                    Nl += 1
                    Dvec[i] = sys.maxsize
                    break
            # finding the new min waiting time
            tl = sys.maxsize
            for i in range(person_served+1, len(Wvec)):
                tl = min(tl, Wvec[i])
            # What if this person was to be served? Since he is no
more, need to assign serving to the next person?
            # Done when you go to the departure section
        else: # this has to be a departure (completion of a service)
            assert(te == td)
            t = te
            for i in range(person_served+1, len(Dvec)):
                if(Dvec[i] == t):
                    # Why not just assign td here? Since we don't know
which person is being served, so who (which index) to assign is not
clear

                    # Not the problem with original model (infinite
waiting) since there every person (who came) will be there waiting in
the queue and so
                    # will have a departue time assigned; but here,
not everyone is waiting, some might have already lost by the time this
time (td) came,
                    # so, this time of departure corresponds to the
person who is still waiting in the queue (Wvec[index]] < infinity) and

                    # has this time of departue. Where do we actually
assign this time of departue? In the next loop.
```

```
                    # In the original model, Nd ++ => Nd corresponded
to the arrival index of the person (and so D[Nd] = t made sende) but
                    # not the case here since some people might have
arrived but got lost,
                    # so Nd ++ will still be less than the arrival
index of the person, so can't do D[Nd] = t
                    # (Note that we always want timestamps to be
mapped to the arrival index of the person for correct and systematic
analysis;
                    # anyone who has arrived atleast once will have
some data corresponding to him or her)

                    # ith person is being served
                    person_served = i
                    n -= 1
                    Nd += 1
                    break
            if(n > 0): # people still there in the queue
                # To find the min of waiting times in the remaining
people again (n of them)
                tl = sys.maxsize
                for i in range(person_served+1, len(Dvec)):
                    tl = min(tl, Wvec[i])
                # To assign a departure time to the next person
waiting in line (still waiting => Wvec[that person] != infinity)
                delta = generate_service_time_for_next_job()
                td = t + delta
                for i in range(person_served+1, len(Dvec)):
                    if(Wvec[i] != sys.maxsize):
                        # (Wvec[i] finite => the person is still
waiting (the person hasn't left yet), so can serve him or her)

                        Dvec[i] = td
                        break
            else:
                tl = sys.maxsize
                td = sys.maxsize
        # Loop invariants
        assert(Na == n + Nl + Nd) # 1
        # 2: If Wvec[i] = infinity, Dvec[i] = infinity
```

Q11

```
def simulate_insurance(a0, n0, T, lmda_arrival, lmda_claim,
lmda_leave, claim_amount_distribution, rate_premium_payment):
    '''
    Simulating an insurance model
    Args:
        a0 (float): initial asset of the company
```

```
        n0 (int): Initial number of customers to the company
        T (float): Time till which to check if the company is solvent
        lmda_arrival (float): rate at which a new customer arrives
        lmda_claim (float): rate at which a claim is made
        lmda_leave (float): rate at which an existing customer leaves
        claim_amount_distribution: distribution for the claim amount
        rate_premium_payment (float): rate of payment of insurance
premiums
    '''
    def generate_time_for_next_event(lmda):
        u = np.random.rand()
        return 1/lmda*math.log(1/u)

    def generate_claim_amount(claim_amount_distribution):
        try:
            if(isinstance(claim_amount_distribution, Number) and
claim_amount_distribution > 0):
                # if it is a positive number, the distribution is
assumed to be a poisson process with that rate, so generate
accordingly
                rate = claim_amount_distribution
                u = np.random.rand()
                return 1/rate*math.log(1/u)
        except Exception as e:
            raise Exception(f"{e}: Other forms of distribution not
supported yet!")

    def generate_discrete_pmf(prob):
        l = len(prob)
        prob = prob/prob.sum()
        u = np.random.rand()
        val = 0
        for i in range(l):
            val = val + prob[i]
            if(u<val):
                return i+1

    t = 0
    a = a0
    n = n0
    delta = generate_time_for_next_event(lmda_arrival + n*lmda_leave +
n*lmda_claim)
    te = t + delta

    while(True):
        if(te>T):
            survived = 1
            break
        else:
```

```
            a = a + n*rate_premium_payment*(te-t)
            t = te
            j = generate_discrete_pmf(np.array([lmda_arrival,
n*lmda_leave, n*lmda_claim])) # using exponential (nu, nv, lmda)
            if(j==1):
                n = n+1
            elif(j==2):
                n = n-1
            else: # j == 3
                assert(j==3)
                claim_amount =
generate_claim_amount(claim_amount_distribution)
                if(claim_amount > a):
                    survived = 0
                    break
                a = a - claim_amount
            delta = generate_time_for_next_event(lmda_arrival +
n*lmda_leave + n*lmda_claim)
            te = t + delta
    return survived

lmda_claim = 10 # per day (this is basically n*lmda_claim of the
algorithm; for correctness, we put n = 1)
claim_amount_distribution = 1/1000 # so that mean = 1000
rate_premium_payment = 11000 # per day (this is n*rate_payment; for
correctness, we put n = 1)
a = 25000
T = 365

# In this question, we assume that 11000 is the total rate of payment
per day i.e. total payment made by all customers per day (absorbs n))
# Further, 10 is the net rate of claims, considering all the current
customers per day
num_iters = 200
output = 0
for itr in range(num_iters):
    survived = simulate_insurance(a, 1, T, 0, lmda_claim, 0,
claim_amount_distribution, rate_premium_payment)
    output += survived

prob_survival = output/num_iters
print("Probability of survival of the firm:", prob_survival)

Probability of survival of the firm: 0.92
```

Q15

```
def simulate_shock_failures(C, alpha, lmda_shock_arrival):
    '''
    Simulate a system with shocks (arriving as a poisson process) that
```

```
have exponential decay
    Args:
        C (float): shock bearing capacity of the system
        alpha (float): rate of decay of the shocks
        lmda_shock_arrival(float): (poisson) rate at which shocks
arrive
    '''
    # helper functions
    def generate_time_interval_to_next_shock(lmda):
        u = np.random.rand()
        return 1/lmda*math.log(1/u)

    def generate_intensity_of_next_shock():
        def f(x):
            return x*math.exp(-x)
        def g(x):
            return math.exp(-x/2)/2
        c = f(2)/g(2) # max of f/g (at x* = 2)
        while(True):
            u1 = np.random.rand()
            y = 1/(1/2)*math.log(1/u1) # lambda = 1/2 => y belongs to
(or comes from the distribution) g(x)
            u2 = np.random.rand()
            if(u2<=f(y)/(c*g(y))):
                return y

    # State variables
    t = 0
    n = 0
    tvec = [0]
    xvec = [0]

    # event loop
    while(True):
        total_shock = 0
        # print(n, xvec, tvec)
        for i in range(1, n+1):
            total_shock += xvec[i]*math.exp(-alpha*(t-tvec[i]))
        # print(total_shock)
        if(total_shock>C):
            # system failed => get out of the loop (stop)
            break
        # time of next shock
        delta =
generate_time_interval_to_next_shock(lmda_shock_arrival)
        te = t + delta
        # Intensity of next shock
        xe = generate_intensity_of_next_shock()
        # Update state variables
```

```
        t = te
        n += 1
        tvec.append(te)
        xvec.append(xe)

    return t # time at which the loop broke (or the system failed)

def run_shock_failures(k, C, alpha, lmda_shock_arrival):
    '''
    Run k (shock failure) simulations to find the expected time of
failure of the system
    Args:
        k (int): number of iterations
        C (float): shock bearing capacity of the system
        alpha (float): rate of decay of the shocks
        lmda_shock_arrival (float): (poisson) rate at which shocks
arrive
    '''
    time_to_fail = 0
    for _ in range(k):
        time_to_fail += simulate_shock_failures(C, alpha,
lmda_shock_arrival)
    time_to_fail = time_to_fail/k
    return time_to_fail
```

(c)

```
k = 100000
C = 0
alpha = 0.5
lmda_shock_arrival = 1
average_time_to_fail = run_shock_failures(k, C, alpha,
lmda_shock_arrival)
print("average time at which the system fails:", average_time_to_fail)
```

```
average time at which the system fails: 0.9965029314662266
```

(d)

```
k = 1000
C = 5
alpha = 0.5
lmda_shock_arrival = 1
average_time_to_fail = run_shock_failures(k, C, alpha,
lmda_shock_arrival)
print("average time at which the system fails:", average_time_to_fail)
```

```
average time at which the system fails: 5.096164898803555
```

Q16

```
def simulate_free_channels_or_else_lost(lmda, T):
    '''
    Simulate 3 channel system with 0 waiting time and weather
dependent service times
    Args:
        lmda (float): rate of job arrival
        T (float): The cutoff time of the system (for system analysis)

    '''
    def generate_new_arrival(lmda):
        # the arrival is assumed to be poisson @lmda
        u = np.random.rand()
        return 1/lmda*math.log(1/u)

    def generate_service_time(weather):
        u = np.random.rand()
        if(weather == 1): # good
            # F(x) = x
            x = u
        else: # bad weather
            # F(x) = x**3
            x = math.pow(u, 1/3)
        return x

    def get_new_timestamp_of_completion(t, w):
        Is = generate_service_time(w) # run service time distributions
here
        timestamp_of_service_completion = t + Is
        return timestamp_of_service_completion

    def run_weather_loop(t):
        if(t%3<2):
            return 1 # good weather
        else:
            return 0 # bad weather

    # Initialize (state) variables
    t = 0
    Na = 0 # number of arrivals till now
    n = 0 # number of messages that got processed till now
    Nl = 0 # number of messages lost till now (invariant: Na = n + Nl
always)
    c1 = 0 # whether channel 1 is busy (1) or free (0)
    c2 = 0 # whether channel 2 is busy (1) or free (0)
    c3 = 0 # whether channel 3 is busy (1) or free (0)
    w = 1 # Whether the weather is good (1) or bad (0)

    # Initialize (event) time variables
```

```
    delta = generate_new_arrival(lmda)
    ta = t + delta
    ts1 = sys.maxsize # time of service completion of the message in
channel 1 (if no message, defaulted to infinity)
    ts2 = sys.maxsize # similar for channel 2 and 3
    ts3 = sys.maxsize

    # Note that if none of the channels are free, it does not matter
whether the weather is good or bad, the message will be lost anyways!
    while(True):
        # What's the next event that will be happening? te represents
the time of next event
        # print(ta, ts1, ts2, ts3)
        te = min(ta, ts1, ts2, ts3)
        if(te > T):
            break
        if(te == ta):
            # a new message arrives
            t = te
            Na += 1
            w = run_weather_loop(t)
            if(c1 == 0): # channel 1 free?
                n += 1
                c1 = 1
                ts1 = get_new_timestamp_of_completion(t, w)
            elif(c2 == 0): # channel 2 free?
                n += 1
                c2 = 1
                ts2 = get_new_timestamp_of_completion(t, w)
            elif(c3 == 0): # channel 3 free?
                n += 1
                c3 = 1
                ts3 = get_new_timestamp_of_completion(t, w)
            else: # no channel free => message is dropped
                Nl += 1
            assert(Na == n + Nl)
            delta = generate_new_arrival(lmda)
            ta = t + delta
        elif(te == ts1): # channel 1 gets free
            t = te
            c1 = 0
            ts1 = sys.maxsize # once channel is free => no message
there => ts = infinity
        elif(te == ts2): # channel 2 gets free
            t = te
            c2 = 0
            ts2 = sys.maxsize
        else:
            assert(te == ts3) # channel 3 gets free
            t = te
```

```
            c3 = 0
            ts3 = sys.maxsize

        # Note that here there is no queue unlike the server systems.
        # Hence, when a channel gets free, it stays free until a new
arrival enters it.
        # This mechanism is unlike server systems where since people
were waiting in a queue and,
        # once a channel gets free, it immediately gets assigned a job
and its processing time is determined
        # henceforth. In this way, we can also compute expected time
for which each of the channels stay free.

    return Na, n, Nl
```

(d)

```
# Initialization
lmda = 2
T = 0

# Auxillary variables
num_iters = 1000
average_number_of_messages_lost = 0

# event loop
for _ in range(num_iters):
    Na, n, Nl = simulate_free_channels_or_else_lost(lmda, T)
    average_number_of_messages_lost += Nl

average_number_of_messages_lost /= num_iters
print("average_number_of_messages_lost:",
average_number_of_messages_lost)
```

```
average_number_of_messages_lost: 0.0
```

(e)

```
# Initialization
lmda = 2
T = 100

# Auxillary variables
num_iters = 10000
average_number_of_messages_arrived = 0
average_number_of_messages_processed = 0
average_number_of_messages_lost = 0

# event loop
for _ in range(num_iters):
    Na, n, Nl = simulate_free_channels_or_else_lost(lmda, T)
```

```
    average_number_of_messages_arrived += Na
    average_number_of_messages_processed += n
    average_number_of_messages_lost += Nl

average_number_of_messages_arrived /= num_iters
print("average_number_of_messages_arrived:",
average_number_of_messages_arrived)
average_number_of_messages_processed /= num_iters
print("average_number_of_messages_processed:",
average_number_of_messages_processed)
average_number_of_messages_lost /= num_iters
print("average_number_of_messages_lost:",
average_number_of_messages_lost)
```

```
average_number_of_messages_arrived: 200.1733
average_number_of_messages_processed: 182.8721
average_number_of_messages_lost: 17.3012
```

Q17

```
def generate_gaussian_variables_by_box_muller_method(mu, sigma):
    '''
    Generating 2 normally distributed independent random variables
using box muller method
    Args:
        mu (float): mean (loc) of the random variables
        sigma (float, > 0): standard deviation (scale) of the random
variables
    Returns:
        random variables (x, y) with the required distribution
    '''
    while(True):
        u1 = np.random.rand()
        v1 = 2*u1 - 1
        u2 = np.random.rand()
        v2 = 2*u2 - 1
        s = v1**2 + v2**2
        if(s<=1):
            break
    x = math.sqrt(-2*math.log(s)/s)*v1
    x = x*sigma + mu
    y = math.sqrt(-2*math.log(s)/s)*v2
    y = y*sigma + mu
    return x, y

# Helper functions
def phi(x):
    y = 1/(1+0.33267*x)
    a1 = 0.4361836
    a2 = -0.1201676
```

```
    a3 = 0.9372980
    return 1-1/math.sqrt(2*math.pi)*(a1*y + a2*y*y +
a3*y*y*y)*math.exp(-x*x/2)

# Initialization
N = 20 # days
S0 = 100
K = 100
mu = -0.05
sigma = 0.3
alpha = mu+sigma**2/2 # < 0 => hence, strategy of section 7.8 makes
sense!

# Auxillary variables
num_iters = 100
value_option = 0
# Locks on gaussian variables for price updates
used_x = True
used_y = True

for itr in range(num_iters):
    price = S0
    payoff = 0
    for m in range(N, 0, -1):
        exercise = True
        exercise = exercise and (price > K)
        if(exercise == True):
            for i in range(1, m+1):
                b = (i*mu-math.log(K/price))/(sigma*math.sqrt(i))
                exercise = exercise and (price > K +
price*math.exp(i*alpha)*phi(sigma*math.sqrt(i) - b) - K*phi(b))
                if(exercise == False):
                    break
        if(exercise == True):
            # we should exercise the option (i.e. buy the stock on
this day (when there are m days to go))
            payoff = price - K
            break
        # Since we use box muller method to generate gaussian RV, they
are generated 2 in number everytime:
        # So, this is to ensure that both are used (once and only
once): one in this iteration, another in the next and then we generate
new ones
        if(used_x and used_y):
            x, y =
generate_gaussian_variables_by_box_muller_method(mu, sigma)
            used_x = False
            used_y = False
        if(used_x == False):
            price = price*math.exp(x)
```

```
            used_x = True
        elif(used_y == False):
            price = price*math.exp(y)
            used_y = True
    value_option += payoff
value_option = value_option/num_iters
print("Value of owning the option:", value_option)
```

```
Value of owning the option: 39.35030208689541
```